

Tema 5: Tratamiento de datos

Inserción, modificación y borrado
de datos

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

Contenido

1. Introducción	2
2. Insert	2
2.1. insert y select	3
3. Update	6
3.1. update y select	7
4. Delete / Truncate	8
4.1. delete y select	8
5. Actividades	9

1. Introducción

En este tema se van a cubrir las sentencias SQL que permiten modificar datos en la base de datos (DML).

2. Insert

La sentencia `insert` permite insertar registros en una tabla, su sintaxis es:

```
INSERT INTO nombre_tabla (nombre_columna1, nombre_columna2, . . . )
VALUES (valor1_columna1, valor1_columna2, ...),
VALUES (valor1_columna1, valor1_columna2, ...),
...;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

Por ejemplo, si tenemos la tabla `planetas` definida como:

```
CREATE TABLE planetas (
  id int NOT NULL AUTO_INCREMENT,
  nombre varchar(50) NOT NULL,
  distancia_tierra decimal(10,0) NOT NULL,
  CONSTRAINT pk_planetas PRIMARY KEY (id),
  CONSTRAINT chk_distnacia CHECK (distancia_tierra > 0));
```

Para introducir valores la sentencia `insert` seria:

```
insert into planetas (nombre, distancia_tierra)
values
('Mercurio', 77000000),
('Venus', 40000000),
('Marte', 225000000);
```

En la sentencia anterior se puede observar que no se ha especificado la columna `id` para dejar al MySQL que la calcule, pero se podría de especificar y meter el valor que deseemos.

2.1. insert y select

Se puede realizar una inserción de datos que provengan de una consulta, la sintaxis es:

```
INSERT INTO nombre_tabla (nombre_columna1, nombre_columna2, . . . )
SELECT nombre_columna1, nombre_columna2, ....
from
...;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

El número de columnas del `insert` y `select` tiene que ser igual. Los datos de ambas columnas también tienen que ser coherentes.

Por ejemplo, si tenemos la siguiente tabla:

```
create table cuerpos_celestes (
    id int primary key auto_increment,
    nombre varchar(50) not null,
    tipo_cuerpo varchar(50),
    distancia_media_km bigint, -- usado para objetos del sistema solar
    distancia_años_luz decimal(15, 6), -- usado para objetos lejanos
    unidad_medida varchar(20),
    descripcion text
);

insert into cuerpos_celestes (nombre, tipo_cuerpo, distancia_media_km,
distancia_años_luz, unidad_medida, descripcion)
values

('Luna', 'Satélite Natural', 384400, 0, 'Kilómetros', 'El único satélite
natural de la Tierra. '),

('Venus', 'Planeta', 41000000, 0, 'Kilómetros', 'Planeta más cercano a la
Tierra. '),

('Sol', 'Estrella', 149600000, 0, 'Kilómetros', 'Centro de nuestro sistema
solar (1 UA). '),

('Marte', 'Planeta', 225000000, 0, 'Kilómetros', 'Conocido como el planeta
rojo. '),

('Júpiter', 'Planeta', 778000000, 0, 'Kilómetros', 'El planeta más grande del
sistema solar. '),

('Plutón', 'Planeta Enano', 5900000000, 0, 'Kilómetros', 'Situado en el
cinturón de Kuiper. '),
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

```
('Próxima Centauri', 'Estrella', NULL, 4.240000, 'Años Luz', 'La estrella más cercana al Sol.'),
```

```
('Sirio', 'Estrella', NULL, 8.600000, 'Años Luz', 'La estrella más brillante del cielo nocturno.'),
```

```
('Pléyades', 'Cúmulo Estelar', NULL, 444.000000, 'Años Luz', 'También conocidas como las Siete Hermanas.'),
```

```
('Andrómeda', 'Galaxia', NULL, 2537000.000000, 'Años Luz', 'La galaxia espiral más cercana a la Vía Láctea.');
```

Se podría hacer un `insert` a la tabla `planetas` de la siguiente manera:

```
insert into planetas (
nombre,
distancia_tierra)
select
nombre,
distancia_media_km
from
cuerpos_celestes
where tipo_cuerpo='Planeta';
```

De hecho, se puede duplicar una tabla (sin sus restricciones) con sus datos en una sola sentencia, esto se suele hacer como `backup`.

```
create table nombre_tabla as
select *
from tabla1;
```

Por ejemplo:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

```
create table backup_planetas as
select * from planetas;
```

3. Update

La sentencia `update` permite actualizar registros en una tabla, su sintaxis es:

```
UPDATE nombre_tabla
SET nombre_columna1=valor,
nombre_Columna2=valor2,
...
WHERE <condicion>;
```

Valor puede ser un literal o puede ser el dato de una columna de la tabla.

Condición puede ser que una columna sea igual a un valor, mayor o igual, que comience por una letra, etc. También se pueden referenciar literales o valores de otra columna.

Siguiendo con la tabla anterior, si se desea actualizar la distancia por ejemplo de Venus a La Tierra se debería ejecutar:

```
update planetas
set distancia_tierra=77500000
where nombre='Venus';
```

Es posible que en alguna situación se deseen actualizar todos los registros, por ello la condición no es un dato obligatorio en la sentencia `update`, por ejemplo si deseamos sumar 100000 a la distancia a La Tierra de cada planeta:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

```
update planetas
set distancia_tierra=distancia_tierra+100000;
```

3.1. update y select

La condición se puede especificar una subconsulta **siempre y cuando se referencie a otra tabla**. Por ejemplo, esta sentencia es correcta:

```
update planetas
set nombre=upper(nombre)
where nombre in
    (select nombre
     from cuerpos_celestes
     where nombre like 'V%');
```

En cambio, esta es incorrecta por referenciar la consulta a la misma tabla que está consultando:

```
update planetas
set nombre=upper(nombre)
where nombre in
    (select nombre
     from planetas
     where nombre like 'V%');
```

```
ERROR 1093 (HY000): You can't specify target table 'planetas' for update in
FROM clause
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

4. Delete / Truncate

La sentencia `delete` permite eliminar registros en una tabla, su sintaxis es:

```
DELETE FROM nombre_tabla
WHERE <condicion>;
```

Siguiendo con la tabla anterior, si se desea eliminar los planetas que se encuentren a más de 100000 km de La Tierra:

```
delete from planetas
where distancia_tierra>=100000;
```

También es posible que se desee borrar todos los registros, en este caso hay 2 opciones:

- `delete` sin condición:

```
delete from planetas;
```

- `truncate`: es la sentencia específica para borrar todos los datos de una tabla (no admite condiciones). Es mucho más rápida que `delete` ya que `delete` va comprobando si se mantiene la integridad con otras tablas (`foreign keys`, etc.), hace borrada en cascada (si la `foreign key` es del tipo `delete cascade`), etc. En cambio, `truncate` no se puede utilizar si la tabla tiene referencias con otras tablas, es decir, `truncate` no comprueba registro a registros, sino que directamente da error si existe cualquier referencia.

```
truncate planetas;
```

4.1. delete y select

En la condición se puede especificar una subconsulta **siempre y cuando se referencie a otra tabla**, es decir, funciona de la misma forma que con un `update`.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

5. Actividades

Actividad 1

Dadas las siguientes tablas y datos:

```
create table clientes (
    id_cliente int primary key,
    nombre varchar(50),
    ciudad varchar(50),
    activo boolean
);

create table pedidos (
    id_pedido int primary key,
    id_cliente int,
    total decimal(10,2),
    fecha date,
    foreign key (id_cliente) references clientes(id_cliente)
);

insert into clientes values
(1, 'Ana', 'Madrid', TRUE),
(2, 'Luis', 'Barcelona', TRUE),
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

```
(3, 'Carlos', 'Madrid', TRUE),
(4, 'Sofia', 'Valencia', TRUE);
```

```
insert into pedidos values
(1, 1, 500.00, '2025-01-10'),
(2, 1, 200.00, '2025-01-15'),
(3, 2, 150.00, '2025-01-12'),
(4, 3, 50.00, '2025-01-20');
```

- Crear la siguiente tabla e inserta en ella los clientes cuyo total de compras sea mayor al promedio de compras de todos los clientes.

```
create table clientes_premium (
    id_cliente int,
    nombre varchar(50),
    total_compras decimal(10,2)
);
```

- Poner clientes a inactivos (actualizar a `activo` a `FALSE`) para aquellos cuyo total de compras sea menor al promedio general de compras.
- Eliminar los pedidos de clientes que estén inactivos.
- Crear una tabla similar a `clientes` e insertar en ella los clientes que no tengan pedidos.
- Aumentar un 10% el campo `total de pedidos` de clientes de Madrid.
- Eliminar clientes que nunca hayan hecho un pedido.

Actividad 2

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

Dadas las siguientes tablas y datos:

```
create table usuarios (
```

```
    id_usuario int primary key,
```

```
    nombre varchar(50),
```

```
    activo boolean
```

```
);
```

```
create table prestamos (
```

```
    id_prestamo int primary key,
```

```
    id_usuario int,
```

```
    fecha_prestamo date,
```

```
    fecha_devolucion date,
```

```
    foreign key (id_usuario) references usuarios(id_usuario)
```

```
);
```

```
insert into usuarios values
```

```
(1, 'Ana', TRUE),
```

```
(2, 'Luis', TRUE),
```

```
(3, 'Carlos', TRUE),
```

```
(4, 'Sofia', TRUE);
```

```
insert into prestamos values
```

```
(1, 1, '2025-01-01', '2025-01-10'),
```

```
(2, 1, '2025-02-01', null),
```

```
(3, 2, '2025-01-15', '2025-01-20'),
```

```
(4, 3, '2025-02-05', null);
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 5: Tratamiento de datos	Inserción, modificación y borrado de datos

- Crear la siguiente tabla e inserta en ella los usuarios que hayan realizado más préstamos que el promedio de préstamos por usuario.

```
create table usuarios_frecuentes (
    id_usuario int,
    nombre varchar(50),
    total_prestamos int
);
```

- Poner usuarios a inactivos (actualizar a `activo` a `FALSE`) para aquellos que no tengan ningún préstamo activo (`fecha_devolucion` es `null`).
- Eliminar los préstamos de usuarios que estén inactivos.
- Crear una tabla similar a `usuarios` e insertar en ella los usuarios que no hayan hecho préstamos.
- Eliminar usuarios que nunca hayan hecho un préstamo.